コマンドライン | Programming Place Plus　C言語編　第４５章

この章の概要

この章の概要です。

- main関数の定義
- 引数のある main関数
- コマンドライン引数の渡し方
  - Visual Studio のプロパティから指定する (Windows)
  - コマンドプロンプトから実行する (Windows)
  - 実行ファイルへのショートカットを経由する (Windows)
  - ターミナルから実行する (OS X)
- コマンドライン引数の利用法
- リダイレクト
- 文字列を数値に変換する
- 練習問題
- 参考リンク
- 更新履歴

マーケターのための企画立案力養成講座（オンデマ...
プロモーション、商品企画、イベントなどに生きる、アイデ...

ソーシャルリスニング基礎講座（オンデマンド配信）
「ソーシャルリスニング基礎講座」は、ネットユーザーの実...

一段上のセンス・デザインに仕上がるPowerPointデ...
パワーポイントは豊富な文字表現に加えて、図や写真、グラ...

main関数の定義

これまでに登場したすべてのプログラムで、main関数には引数がありませんでした。しかし、実は引数がある main関数を作ることはできます。

main関数の仮引数と戻り値に関して、標準規格は、次の４通りのいずれかの方法で定義しなければならないと定めています。

1. 仮引数が void型で、戻り値型が int型
2. 第１引数が int型、第２引数が char*[]型で、戻り値型が int型
3. 上記１、２と事実上同じ意味になるもの
4. 処理系定義の方法

１は、これまでどおりの main関数のことです。つまり、次のように定義します。

```
int main(void)
{
    return 0;
}
```

２は、この章のテーマになるものです。詳細は後の項で解説しますが、ともかく、仮引数を２つ持っています。型も守らねばなりません。仮引数の名前に関しては自由に決められますが、慣例的に argc、argv という名前を使うことが多く、それ以外の名前を付ける理由は特にないはずです。

具体的には、次のようになります。

```
int main(int argc, char* argv[])
{
    return 0;
}
```

３は、上記の２つと同じになる別の書き方を許すということです。たとえば、「int」の部分を、typedef で作った別名にしてもいいとか、char*[] を char** と記述してもいいといったことです。後者はわりとよく見かけます。

```
int main(int argc, char** argv)
{
    return 0;
}
```

先頭へ戻る

４は、処理系定義の記法があるのなら、それを使っても良いということです。

戻り値型の指定を省略して、以下のように書いているプログラムを見かけることがあります。

```
main(void)
{
    return 0;
}
```

これで問題がなかったのは、関数宣言で戻り値型の指定を省略すると int型とみなされるルールがあったためです。しかし、このルールは C99 で撤廃されているため、現在では正しくありません（第９章）。

引数のある main関数

では、main関数の書き方のうち２番目のパターンについて、詳細を見ていきましょう。

```
int main(int argc, char* argv[])
{
    return 0;
}
```

繰り返しになりますが、仮引数の名前は変えても構いませんが、変える理由はありません。また、型名は同じ意味になるのなら変えても構いません。第２引数の型を「char**」にしているプログラムはよく見かけます。

main関数は、プログラムの実行を開始したとき、最初に呼び出される関数であって、プログラム内には main関数を呼び出している箇所はありません。では誰が実引数を指定しているのかというと、プログラムを実行する環境自身です。

Visual Studio とか Xcode のような IDE でプログラムを実行していると気づきませんが、普通、作った実行ファイルは、コマンドプロンプトやターミナルなどから実行します。

IDE を使うのは、プログラムを書いている開発者だけです。アプリケーションを入手して、それを実行するユーザーは、Windows や macOS の環境上で実行するはずです。このとき、その実行ファイルに対してパラメータを渡すことができ、それが main関数の仮引数へ引き渡されていきます。このような引数を、コマンドライン引数（コマンド引数）と呼びます。

前の章で、system関数を説明したときに、以下のような呼び出し例を紹介しました。

```
system( "copy /B test.txt test_copy.txt" );  // Windows
system( "cp test.txt test_copy.txt" );       // macOS
```

この場合、「copy」や「cp」が実行ファイルの名前です。そして、それぞれの後ろにある「/B」「test.txt」「test_copy.txt」といった部分が、コマンドライン引数です。

copy や cp というプログラムは、コマコマンドラインンド引数を受け取って、それを使って処理を行うように作られているので、実行の際に、こうしてコマンドライン引数を指定する必要があります。

では実際に、コマンドライン引数を使えるプログラムを書いてみましょう。

```
#include <stdio.h>

int main(int argc, char* argv[])
{
    for( int i = 0; i < argc; ++i ){
        puts( argv[i] );
    }

    return 0;
}
```

main関数の第２引数の型は char*[] ですから、文字列を要素とする配列です。実際には、いつものように、配列の先頭を指すポインタとして渡されてきます。

argv[0] には、プログラムの名前が入ります。プログラムの名前とは具体的に何であるのかという規定はありませんが、通常、実行ファイルの名前から拡張子を抜いたものになっていると思われます。ただし、環境によっては、argv[0] の中身が "" となることがあります。そのため、移植性を考えるなら、ここに入ってくる情報にはあまり期待が持てません。

argv[1]～argv[argc-1] には、コマンドライン引数がそれぞれ指定された順番どおりに格納されています。ここから分かるように、仮引数argc に格納されているのは、仮引数argv の要素数です。 なお、argv[argc] にはヌルポインタが入っていることが保証されています。

このプログラムを実行する際には、コマンドライン引数を指定しなければなりません。次の項で、その方法を説明します。

コマンドライン引数の渡し方

プログラムを実行する際に、コマンドライン引数を渡すには、次のようにいくつかの方法があります。先ほどのサンプルプログラムを例に試してみます。なお、プログラムの名前は "test" であるとします。

Visual Studio のプロパティから指定する (Windows)

先頭へ戻る

Visual Studio上で実行したい場合は、こちらのページを参照してください。

コマンドプロンプトから実行する (Windows)

コマンドプロンプトを起動して、そこから実行するという方法があります。むしろ、この方法が基本と言えます。

Windows のスタートメニューから、「ファイル名を指定して実行」を選択し、「名前」の欄に「cmd」と入力して決定します。これでコマンドプロンプトが起動します（方法はいくつもありますが、ともかくコマンドプロンプトが開けばよいです）。

ここから、cdコマンドを使って、実行ファイルがあるフォルダまで移動します。

```
C:¥Users¥master>cd "C:¥myprogram¥Debug"
```

そして、次のようにプログラムを実行します。

```
C:¥myprogram¥Debug>test xyz 100
```

今回、プログラムの名前を "test" と仮定しています。自分のプログラムの名前（.exeファイルの拡張子を抜いた名前）に置き換えてください。

複数のコマンドライン引数を指定する場合は、それぞれの間を半角スペースで空けるようにして、たとえば、「xyz 100」のように書きます。半角スペースを含んだ文字列を指定したければ、「"xyz 100"」のように、全体を "" で囲んでください。

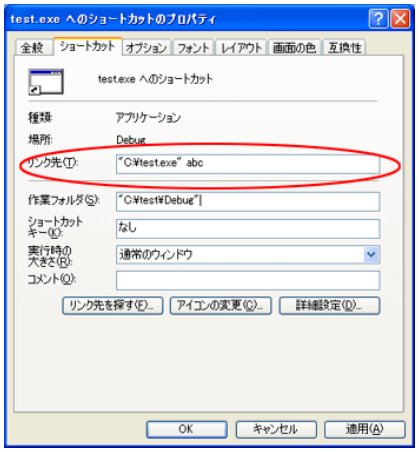
「xyz 100」を設定してから実行すると、次のような結果になります。

実行結果

```
test
xyz
100
```

実行ファイルへのショートカットを経由する (Windows)

Windows のエクスプローラーから、実行ファイルのショートカットを作成し、ショートカットの方のプロパティを開きます。そして、「リンク先」という部分の最後尾に、コマンドライン引数を記述します。



これで、ショートカットの方からプログラムを実行すれば、コマンドライン引数を渡せます。

ターミナルから実行する (OS X)

macOS の場合は、ターミナルを起動して、そこから実行できます。ターミナル上で、cdコマンドを使って、実行ファイルのあるフォルダまで移動します。

```
username$ cd Desktop/myprogram
```

そして、次のようにプログラムを実行します。

```
myprogram username$ ./test xyz 100
```

今回、プログラムの名前を "test" と仮定しています。自分のプログラムの名前（.exeファイルの拡張子を抜いた名前）に置き換えてください。拡張子がないと、単に test と指定するだけでは認識されませんが、名前の前に ./ をつけてカレントディレクトリにある test であることを明示すると、実行できます。

複数のコマンドライン引数を指定する場合は、それぞれの間を半角スペースで空けるようにして、たとえば、「xyz 100」のように書きます。半角スペースを含んだ文字列を指定したければ、「"xyz 100"」のように、全体を "" で囲んでください。

「xyz 100」を設定してから実行すると、次のような結果になります。

実行結果

```
test
xyz
100
```

先頭へ戻る

高精度形状測定機「LUPHOScan」

スピンドル上の光学部品を回転させながらスキャン測定することで真形状を3Dで測定

テーラーホブソン

コマンドライン引数の利用法

プログラムをするときに、コマンドライン引数を渡せるようになれば、これまでに取り上げてきたファイル処理と組み合わせて、より実用性の高いプログラムが作成できます。

ここでは、コマンドライン引数からファイル名を渡し、そのファイルの内容を標準出力に出力するプログラムを作成してみます。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if( argc < 2 ){
        fputs( "コマンドライン引数が不足しています。¥n", stderr );
        exit( EXIT_FAILURE );
    }

    FILE* fp = fopen( argv[1], "r" );
    if( fp == NULL ){
        fprintf( stderr, "%s のオープンに失敗しました。¥n", argv[1] );
        exit( EXIT_FAILURE );
    }

    for( ;; ){
        char buf[80];

        if( fgets( buf, sizeof(buf), fp ) == NULL ){
            if( feof(fp) ){
                break;
            }
            else{
                fputs( "ファイルの読み取りに失敗しました。¥n", stderr );
                exit( EXIT_FAILURE );
            }
        }

        // 末尾の改行文字を取り除く
        char* p = strchr( buf, '¥n' );
        if( p != NULL ){
            *p = '¥0';
        }

        puts( buf );
    }

    if( fclose( fp ) == EOF ){
        fputs( "ファイルクローズに失敗しました。¥n", stderr );
        exit( EXIT_FAILURE );
    }

    return 0;
}
```

入力ファイル (test.txt)

```
aaaaa
bb  b

dddddd
```

実行結果

```
aaaaa
bb  b

dddddd
```

コマンドライン引数には "test.txt" を指定しています。

コマンドライン引数を扱う場合、想定した個数の引数がきちんと渡されてきていることを確認するようにしましょう。今回のサンプルプログラムであれば、コマンドライン引数でファイル名が渡されることを想定しているので、main関数の仮引数argc は 2 になっているはずです。もし、argc が 2未満であればエラーとしています。

逆に、コマンドライン引数が多すぎるというケースもあり得ますが、今回は単純に無視しています。これも必要に応じて、処置を施しましょう。

また、今回のようにファイル名が渡されることを想定していても、100 のような整数が渡されるかもしれません。そういった、誤った入力があっても正しく動作する（あるいはエラーとして処理する）ようなプログラムを書かなければいけません。今回は、ファイル名として正しくない入力は、fopen関数を失敗させるので、単に、ファイルが開けないというエラーになっています。

リダイレクト

先頭へ戻る

先ほどのサンプルプログラムは、コマンドライン引数からファイル名を受け取って、内容を出力しています。これは便利なようにも思えますが、ファイル名を受け取ることにしたが故に、標準入力の内容を出力するという目的には使えません。

プログラムの内容次第ですが、ファイルと標準入出力とを区別する必要がないのであれば、無意味に、ファイル専用のプログラムにするのはもったいないかもしれません。そのような場合は、プログラム自体は標準入出力を相手にできるように作ると良いかもしれません。

標準入力の内容を出力するように書き換えると次のようになります。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
    for( ;; ){
        char buf[80];

        if( fgets( buf, sizeof(buf), stdin ) == NULL ){
            if( feof(stdin) ){
                break;
            }
            else{
                fputs( "ファイルの読み取りに失敗しました。¥n", stderr );
                exit( EXIT_FAILURE );
            }
        }

        // 末尾の改行文字を取り除く
        char* p = strchr( buf, '¥n' );
        if( p != NULL ){
            *p = '¥0';
        }

        puts( buf );
    }

    return 0;
}
```

実行結果

```
aaaaa   <-- 入力した内容
aaaaa
bb  b   <-- 入力した内容
bb  b
        <-- 入力した内容

dddddd  <-- 入力した内容
dddddd
```

前の項のサンプルプログラムでは、コマンドライン引数で指定したファイルから入力していましたが、これを標準入力から行うように変えただけです。コマンドライン引数を受け取れる形になっていますが、使っていません。

繰り返し入力を求められてプログラムが終了できないかもしれません。Ctrl+Z を入力すると EOF を入力できるので、これで終了できます。

このように標準入力からデータを受け取るように書かれたプログラムは、ソースコードを修正することなく、ファイルからデータを受け取るように変更できます。そのためには、リダイレクトという方法を使います。

リダイレクトとは、プログラムを実行するときに、標準入力ストリームや標準出力ストリームと結びついている先を任意に切り替える機能です。たとえば、標準入力ストリームを、test.txt と結びつけるようにして実行すれば、前の項のサンプルプログラムと同じ結果を得られます。

なお、リダイレクトという機能はC言語のものではなく、コマンドプロセッサが持っているものです。ほとんどのコマンドプロセッサで、この機能が使えるはずです。

このサンプルプログラムは、次のようにコマンドライン引数の指定なしで実行できます。

```
test
```

この場合、標準入力から入力を受け取り、標準出力へ結果を出力します。

リダイレクトによって、標準入力を test.txt に切り替えるには、次のようにします。

```
test < test.txt
```

実行するプログラム名に続けて、< を入力し、さらに切り替え先を入力します。この状態で、先ほどのサンプルプログラムを実行すると、次のような結果を得られます。

入力ファイル (test.txt)

```
aaaaa
bb  b

dddddd
```

実行結果

```
aaaaa
bb  b

dddddd
```

このように、test.txt の内容が出力されます。つまり、stdin が意味するものが、標準入力から test.txt に切り替わっているので、「fgets(buf, sizeof(buf), stdin)」は、test.txt から入力を受け取るようになったのです。

今度は標準出力を、out.txt に切り替えてみましょう。次のように実行します。

先頭へ戻る

```
test > out.txt
```

こうすると、標準入力から入力されたデータは、out.txt へ出力されます。

実行結果

```
aaaaa   <-- 入力した内容
bb  b   <-- 入力した内容
        <-- 入力した内容
dddddd  <-- 入力した内容
```

出力ファイル (out.txt)

```
aaaaa
bb  b

dddddd
```

標準入力と標準出力のリダイレクトを同時に行うことも可能です。

```
test < test.txt > out.txt
```

この場合、標準入力は test.txt に、標準出力は out.txt に結び付けられます。

入力ファイル (test.txt)

```
aaaaa
bb  b

dddddd
```

実行結果

```

```

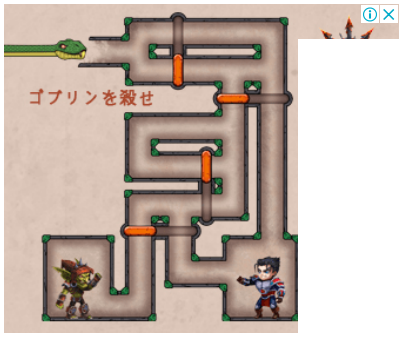
出力ファイル (out.txt)

```
aaaaa
bb  b

dddddd
```

なお、コマンドライン引数も与える場合には、以下のように、先にコマンドライン引数を記述します。

```
test 100 200 < test.txt > out.txt
```





文字列を数値に変換する

コマンドライン引数は文字列として渡されてくるため、数値を扱う場合に困ります。たとえば、１つ目のコマンドライン引数にファイル名を、２つ目の引数に行数を渡すと、その行の内容を出力するプログラムを作るとします。その場合、次のように実行します。

```
test test.txt 7
```

これは、test.txt の 7行目を出力するという意味です。このとき「7」の部分は、argv[2] に入っている訳ですが、これは "7" という文字列になってしまっているので、整数値として扱えるようにする方法を知っておく必要があります。

基本的には、文字列を整数に変換する標準ライブラリ関数を使うだけです。たとえば、atoi関数を使えます。atoi関数は、<stdlib.h> に、以下のように宣言されています。

```
int atoi(const char* str);
```

名前の由来は、"Ascii TO Integer" です。つまり、ASCIIコードの文字列を int型の値に変換します。

> long int型の値に変換して返す atol関数、double型で返す atof関数、long long int型で返す atoll関数があります。

atoi関数は、引数に数字として解釈できるような文字で構成された文字列を渡すと、それを整数値に変換して返します。文字列を先頭から見ていって、数字として解釈できない文字が現れたら、その手前までを変換します。たとえば、"123abc" を指定すると、123 が返されます。

なお、文字列の先頭部分につく符号「+」「-」は、符号として認識されます。また、先頭部分に空白文字がある場合、それは無視されます。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
```

先頭へ戻る

```
{
    if( argc < 3 ){
        fputs( "コマンドライン引数が不足しています。¥n", stderr );
        exit( EXIT_FAILURE );
    }

    int targetLine = atoi(argv[2]);

    FILE* fp = fopen( argv[1], "r" );
    if( fp == NULL ){
        fprintf( stderr, "%s のオープンに失敗しました。¥n", argv[1] );
        exit( EXIT_FAILURE );
    }

    for( int line = 1; ; ++line ){
        char buf[80];

        if( fgets( buf, sizeof(buf), fp ) == NULL ){
            if( feof(fp) ){
                break;
            }
            else{
                fputs( "ファイルの読み取りに失敗しました。¥n", stderr );
                exit( EXIT_FAILURE );
            }
        }

        if( line != targetLine ){
            continue;
        }

        // 末尾の改行文字を取り除く
        char* p = strchr( buf, '¥n' );
        if( p != NULL ){
            *p = '¥0';
        }

        puts( buf );
    }

    if( fclose( fp ) == EOF ){
        fputs( "ファイルクローズに失敗しました。¥n", stderr );
        exit( EXIT_FAILURE );
    }

    return 0;
}
```

入力ファイル (test.txt)

```
aaa
bbb
ccc
ddd
eee
fff
ggg
hhh
```

実行結果

```
ggg
```

ところが atoi関数には問題があります。たとえば、"abc" のように明らかに数字として解釈できない文字列を渡したときに、それと分かるようなエラー値を返してくれません。つまり、エラーの検出が不可能なのです。

お手軽な関数ではありますが、エラーになる可能性がある場面では使うべきではありません。コマンドライン引数は、どんな入力がなされるか分からないので、「エラーになる可能性がある場面」に該当します。

そこで代わりに、strtol関数を使います。strtol関数は、<stdlib.h> に、以下のように宣言されています。

```
long int strtol(const char* restrict str, char** restrict endptr, int radix);
```

restrict については、第５７章で取り上げます。動作に影響はないので、今は無視して問題ありません。

第１引数に、変換元の文字列を渡します。

第２引数には、最初の変換不能な部分のメモリアドレスを受け取るポインタ変数を渡します。

第３引数には、基数を指定します。つまり、文字列に含まれている数字が、何進法で表記されているとみなすかを指示します。

戻り値は、変換結果が返されます。変換がまったく行えなかった場合には 0 が返されます。また、変換結果が long int型で表現できる範囲を超えてしまう場合には、 上限値よりも大きいなら、LONG_MAX が、下限値よりも小さいなら、LONG_MIN が返されます。この場合、errnoに ERANGE がセットされます。errno については後述します。

第２引数が分かりづらいですが、これはたとえば、"123abc" を変換しようとした場合には、"123" は変換可能で、"abc" は変換不能ですから、"abc" の先頭のメモリアドレスを受け取るということです。

atoi関数と比べると途端に複雑になった感がありますが、エラーが検出できるという面から、strtol関数の方が推奨されます。

strtol関数と同種の関数として、unsigned long型に変換する strtoul関数、double型に変換する strtod関数、float型に変換する strtof関数、long double型に変換する strtold関数、long long型に変換する strtoll関数、unsigned long long型に変換する strtoull関数があります。

strtol関数を使って、先ほどのサンプルプログラムを書き換えてみます。

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
```

先頭へ戻る

```c
#include <string.h>

int main(int argc, char *argv[])
{
    if( argc < 3 ){
        fputs( "コマンドライン引数が不足しています。¥n", stderr );
        exit( EXIT_FAILURE );
    }

    char* endPtr;

    errno = 0;
    int targetLine = strtol( argv[2], &endPtr, 10 );
    if( errno == ERANGE || argv[2] == endPtr ){
        fprintf( stderr, "行数の指定が無効です。(%s)¥n", argv[2] );
        exit( EXIT_FAILURE );
    }

    FILE* fp = fopen( argv[1], "r" );
    if( fp == NULL ){
        fprintf( stderr, "%s のオープンに失敗しました。¥n", argv[1] );
        exit( EXIT_FAILURE );
    }

    for( int line = 1; ; ++line ){
        char buf[80];

        if( fgets( buf, sizeof(buf), fp ) == NULL ){
            if( feof(fp) ){
                break;
            }
            else{
                fputs( "ファイルの読み取りに失敗しました。¥n", stderr );
                exit( EXIT_FAILURE );
            }
        }

        if( line != targetLine ){
            continue;
        }

        // 末尾の改行文字を取り除く
        char* p = strchr( buf, '¥n' );
        if( p != NULL ){
            *p = '¥0';
        }

        puts( buf );
    }

    if( fclose( fp ) == EOF ){
        fputs( "ファイルクローズに失敗しました。¥n", stderr );
        exit( EXIT_FAILURE );
    }

    return 0;
}
```

入力ファイル (test.txt)

```
aaa
bbb
ccc
ddd
eee
fff
ggg
hhh
```

実行結果

```
ggg
```

strtol関数のエラーをチェックするとき、最も注意が必要なのは、戻り値が 0 かどうかでは判断できないという点です。 なぜなら、"0" という文字列を変換した場合は、正しい変換の結果として「0」が返されるからです。

まったく変換できなかったことを検出するには、第２引数で受け取ったメモリアドレスを、第１引数に指定したメモリアドレスと比較します。

たとえば、"abc" のような、まったく変換不能な文字列を変換しようとした場合、第２引数には "abc" の先頭のメモリアドレスが返されることになるので、第１引数のメモリアドレスと一致します。

ちなみに、「途中までは変換できたが、途中から変換できなかった」場合をエラーとみなすのかどうかは、検討が必要です。大抵のサンプルプログラムは、これを成功とみなして書かれていますが、場合によっては検出する必要性があるかもしれません。必要ならば、やはり第２引数で受け取ったメモリアドレスを見て、今度は、第１引数の末尾のメモリアドレスが入っていることを調べます。

また、文字列に含まれている数字が巨大すぎて、long int型に収まらないケースも検出しなければなりません。これは、変換自体は成功していても、正しく結果を返せないという状況です。

この場合、strtol関数は、LONG_MAX や LONG_MIN を返す訳ですが、0 を返す場合と同様に、変換結果がたまたま LONG_MAX や LONG_MIN と一致している可能性があるため、エラーチェックには使えません。正しい検出方法は、まず errno に ERANGE が格納されたことを調べることです。

errno は、errno.h に定義されており、一部の標準ライブラリ関数が、内部で起こしたエラーを伝えるために使うもので、グローバル変数であると考えればよいです。また、ERANGE はエラーの種類を表現するオブジェクト形式マクロで、他にもいくつか種類があります。

> 厳密には、errno はグローバル変数ではないこともありますが、意識する必要はありません。

エラーを errno で報告する標準ライブラリ関数を呼び出す際には、その呼び出しの直前で 0 を代入しておき、呼び出しの直後で値を調べるようにします。0 は、エラーが起きていないことを表す値です。

errno に ERANGE が格納されていることを検出した後、上限値を超えているのか、下限値を超えているのかに興味があるのであれば、続けて、戻り値が LONG_MAX なのか LONG_MIN なのかを調べます。

先頭へ戻る

```
errno = 0;
int targetLine = strtol( argv[2], &endPtr, 10 );
if( errno == ERANGE ){
    if( targetLine == LONG_MAX ){
        fprintf( stderr, "行数の指定が上限値を超えています。(%s)¥n", argv[2] );
    }
    if( targetLine == LONG_MIN ){
        fprintf( stderr, "行数の指定が加減値を超えています。(%s)¥n", argv[2] );
    }
    exit( EXIT_FAILURE );
}
else if( argv[2] == endPtr ){
    fprintf( stderr, "行数の指定が無効です。(%s)¥n", argv[2] );
    exit( EXIT_FAILURE );
}
```

こうなると非常に面倒です。これを見てしまうと、atoi関数に戻りたくなる気持ちは理解できますが、やはりエラーの可能性を見過ごせないので、strtol関数をラップしたものを用意しておくなどすると良いかもしれません。

```
/*
    strtol関数をラップしたもの
    引数：
        str:    変換元の文字列。strtol関数の第１引数と同じ。
        radix:  基数。strtol関数の第３引数と同じ。
        result: 変換結果を受け取るポインタ変数。変換に失敗した場合には、何も格納されない。
    戻り値：
        変換が成功したら 0以外、失敗したら 0 が返される。
*/
int my_strtol(const char* str, int radix, long int* result)
{
    assert( str != NULL );
    assert( result != NULL );

    char* end;

    errno = 0;
    long int num = strtol( str, &end, radix );
    if( errno == ERANGE ){
        if( num == LONG_MAX ){
            fputs( "変換結果が上限値を超えた。¥n, stderr );
        }
        if( num == LONG_MIN ){
            fputs( "変換結果が下限値を超えた。¥n, stderr );
        }
        return 0;
    }
    else if( str == end ){
        fputs( "１文字も変換できなかった。¥n", stderr );
        return 0;
    }

    *result = num;
    return 1;
}
```

こうした関数があれば、次のように簡潔に書けます。

```
long int targetLine = 0;
if( my_strtol( argv[2], 10, &targetLine ) == 0 ){
    exit( EXIT_FAILURE );
}
```

先頭へ戻る

練習問題

問題①　コマンドライン引数から、0個以上の整数を受け取り、その合計を標準出力に出力するプログラムを作成してください。

問題②　コマンドライン引数に、

```
test 15 * -3
```

のように、「整数」「演算子」「整数」を渡したとき、全体を計算式とみなして計算結果を標準出力へ出力するプログラムを作成してください。（環境によっては、* の部分がうまく解釈されないかもしれません。その場合は、

```
test 15 '*' -3
```

のように、' ' で囲むことを試してみてください。

問題③　C言語のソースファイルやヘッダファイルを読み込んで、コメント部分を除去した結果を出力するプログラムを作成してください。読み込むファイルの名前は、コマンドライン引数から受け取るようにしてください。

問題④　コマンドライン引数から、ファイルパスを２つ指定し、１つ目のファイルの内容を、２つ目のファイルへ追記するプログラムを作成してください。 たとえば、

```
test in.txt out.txt
```

としたとき、in.txt の内容を、out.txt の末尾へ追記します。

解答ページはこちら。

参考リンク

- Programming Place Plus　C言語編　参考書籍
  -- 当サイトの参考書籍一覧ページ。C言語に関する書籍を多数紹介。
- Programming Place Plus　C言語編　リンク集
  -- 当サイトの参考Webサイト集。C言語の全般的な学習に有益なサイトを紹介。

更新履歴

- '2019/8/20
  - fgets関数を呼び出したあと、改行文字を取り除いて puts関数へ渡すようにサンプルコードを修正
- '2019/8/6
  - 「コンパイラ」よりも「処理系」の方が適切ならば、「処理系」と書くように統一
- '2019/8/1
  - 解説のベースを C99 に上げる対応
    - 標準ライブラリ関数の宣言に restrict を付加
- '2019/7/23
  - 解説のベースを C99 に上げる対応
    - ローカル変数の宣言を、ブロックの先頭以外の位置でも行う
- '2019/7/15
  - 解説のベースを C99 に上げる対応
    - ループ制御変数を for文の初期設定式で宣言するように修正

«さらに古い更新履歴を展開する»

前の章へ　（第４４章　ファイルに対する操作）

次の章へ　（第４６章　マルチバイト文字）

C言語編のトップページへ

Programming Place Plus のトップページへ

はてなブックマーク に保存	Pocket に保存	Facebook でシェア
Twitter でツイート	Twitter をフォロー	LINE で送る
RSS	管理者情報	プライバシーポリシー

先頭へ戻る